

Prompt 7

This Prompt 7 sticks closely to the **original plan's role** for Prompt 7: building the **system prompt constructor** for narration, injecting current location, time of day, present NPCs, inventory/money, and canon-era context into the API call. It also preserves the **optional Gemini + local fallback** architecture established in your current Prompt 6, rather than adding more gameplay systems too early.

KKC TEXT ADVENTURE — PROMPT 7 OF 180

=====

Dynamic Prompt Builder: Location, Time, NPCs, Inventory, Money, Canon-Era Context

=====

This prompt builds directly on Prompts 1, 2, 3, 4, 5, and 6.
Do not restructure, rename, or refactor anything from those prompts.
Only add to what exists.

This prompt follows the original Phase 1 plan role for Prompt 7:
build the dynamic narration prompt constructor that injects current
location, time of day, present NPCs, inventory/money, and canon-era
context into the optional Gemini narration call.

Do not add new gameplay systems in this prompt.

HARD BANS — UNCHANGED

No Anthropic. No OpenAI. No additional AI providers.
Gemini remains the only optional LLM provider in this codebase.

No web server. No frontend. No HTTP libraries beyond what Prompt 6
already introduced for Gemini.

No new commands.

No new game rules.

No moving game truth into the LLM.

No lore registry yet — that belongs in the next prompt.

ARCHITECTURE RULE — STILL ABSOLUTE

engine/* owns:

- command parsing
- movement
- access checks
- time advancement

- state mutation
- persistence
- sympathy truth
- social truth
- music/Eolian truth

narration/* owns:

- prose rendering only
- prompt construction only
- fallback rendering only

The LLM:

- must never mutate PlayerState
- must never write to the DB
- must never invent exits, items, NPCs, outcomes, or lore truth
- must only narrate from structured context supplied by the engine

repl.ts still saves state exactly once per turn.

GOAL OF THIS PROMPT

By the end of Prompt 7, the Gemini narration path should have a proper dynamic prompt builder that:

- injects current location name and base description
- injects time of day and day number
- injects visible NPC names
- injects inventory summary and current money
- injects current academic rank and basic condition
- injects lightweight canon-era context appropriate to the current place in the story
- injects only engine-truth outcomes already resolved by the engine
- produces a clean, compact prompt string for Gemini

The local deterministic path must continue to work unchanged.

SCOPE OF THIS PROMPT

This prompt covers:

1. Extend NarrationSceneContext to include inventory and money summary
2. Build a promptBuilder.ts module
3. Add lightweight era/location context helpers
4. Update GeminiNarrator to use promptBuilder.ts
5. Add tests for prompt construction and fallback safety

This prompt does NOT cover:

- a full canon registry (Prompt 8)
- lore guard / forbidden phrase logging beyond Prompt 6 validation
- mystery registry as a separate module
- NPC freeform LLM conversation
- new commands
- new locations or new NPCs
- new economy or reputation systems

NEW FILES IN THIS PROMPT

Add exactly these files:

```
src/  
  narration/  
    promptBuilder.ts  
tests/  
  promptBuilder.test.ts
```

Modify these existing files:

```
src/types/index.ts  
src/narration/narrationContext.ts  
src/narration/canonInstructions.ts  
src/narration/geminiNarrator.ts
```

Do not add any other files.

TYPES — update src/types/index.ts

Do not remove or rename existing types.

Update NarrationSceneContext to include these additional fields:

```
export interface NarrationSceneContext {  
  command: string;  
  player_summary: {  
    era: string;  
    location_id: string;  
    time_of_day: TimeOfDay;  
    day_number: number;  
    money_drabs: number;  
    academic_rank: AcademicRank;  
    hunger: number;
```

```

    fatigue: number;
    warmth?: number;
    injuries: string[];
};
inventory_summary: {
    item_names: string[];    // e.g. ["Kvothe's lute", "tallow candle x3"]
    money_display: string;    // e.g. "3 talents, 2 drabs"
};
location_summary: {
    id: string;
    name: string;
    description_base: string;
    exits: string[];        // directions only
};
npc_summary: {
    names: string[];
};
canon_era_context: {
    era_label: string;        // e.g. "University era"
    location_group: string;    // e.g. "University", "Imre", "Eolian"
    notes: string[];          // 2–4 compact grounding notes
};
engine_truth: {
    movement_message?: string;
    sympathy_outcome?: string;
    social_outcome?: string;
    music_outcome?: string;
};
}

```

Do not add any other new types in this prompt.

NARRATION CONTEXT — update src/narration/narrationContext.ts

Extend buildNarrationSceneContext(...) to populate the new fields.

Requirements:

Inventory summary:

- item_names should be human-readable
- include quantity suffix only when quantity > 1
- examples:
 - "Kvothe's lute"
 - "tallow candle x3"
 - "iron drab x2"
- keep order stable and deterministic

- if inventory is empty, item_names should be []

Money summary:

- use formatCurrency from renderStatus.ts
- store the result in inventory_summary.money_display

Canon era context:

- build lightweight grounded notes based only on current era and current location cluster / location id
- do not invent lore
- notes must be short, factual, and safe

Required canon_era_context behaviour:

For University-side locations:

era_label: "University era"

location_group: "University"

notes should include points like:

- this is Kvothe's student period at the University
- money, rank, and reputation matter
- sympathy is trained and costly, not casual
- access to places may depend on standing or time

For Stonebridge / crossing:

era_label: "University era"

location_group: "River crossing"

notes should include:

- this is the route between University and Imre
- crossing takes meaningful time in lived experience
- public movement and social visibility matter

For Imre / Eolian-side locations:

era_label: "University era"

location_group: "Imre"

notes should include:

- Imre is distinct from the University in mood and social texture
- the Eolian is a reputation-bearing music venue
- talent pipes must be earned
- this is a public social environment

Rules:

- notes array must contain 2–4 entries
- notes must be deterministic
- do not include unresolved mysteries
- do not include hidden state or blocked exits
- keep it compact

This file remains pure. It does not call the database.

CANON INSTRUCTIONS — update src/narration/canonInstructions.ts

Keep KKC_CANON_INSTRUCTIONS, but strengthen it slightly.

Add the following explicit rules if not already present:

- Use the canon_era_context notes as guardrails, not as a license to invent.
- Visible NPCs may be mentioned; absent NPCs must not be introduced.
- Exits may only be described from the supplied accessible directions.
- Inventory and money may be mentioned only if naturally relevant.
- Do not mention blocked paths, hidden rules, trust scores, or raw mechanics.
- Do not narrate outcomes that differ from engine_truth.
- The narration should make the world feel grounded and socially real, not florid or epic by default.

Do not remove the existing mystery bans.

PROMPT BUILDER — src/narration/promptBuilder.ts

Implement and export:

buildGeminiSystemPrompt(context: NarrationSceneContext): string
buildGeminiUserPrompt(context: NarrationSceneContext): string

Rules:

- These functions produce plain text prompt strings only
- Do not output raw JSON
- Do not include unnecessary formatting noise
- Keep prompts compact and stable

buildGeminiSystemPrompt(context)

This should assemble a single system-style instruction string using:

1. KKC_CANON_INSTRUCTIONS
2. A section titled: "Grounded Scene Context"
3. The current era label and location group
4. The canon_era_context notes as bullet-like lines
5. A section titled: "Authoritative Constraints"
including:
 - only use visible NPCs

- only use accessible exits
- do not invent lore or mystery answers
- honour engine_truth exactly
- keep prose restrained, present tense, second person

Do not include the player inventory list or money display here.
That belongs in the user prompt.

buildGeminiUserPrompt(context)

This should assemble the current playable scene in compact prose-ready form.

Include:

- command
- location name
- base description
- time of day and day number
- visible exits (directions only)
- visible NPC names
- current money display
- inventory item names
- academic rank
- simple condition summary:
 - hunger
 - fatigue
 - warmth if present
 - injuries if any
- engine_truth lines if present

Formatting rules:

- plain text only
- readable sections
- no JSON
- no markdown tables
- no long lists if empty
- if NPC list is empty, say "Visible people: none"
- if inventory is empty, say "Inventory: nothing of note"

End with:

"Write the scene now."

GEMINI NARRATOR — update src/narration/geminiNarrator.ts

Do not refactor working fallback logic.

Only change prompt construction.

Update GeminiNarrator.narrateScene(context) to:

1. build the system prompt using buildGeminiSystemPrompt(context)
2. build the user prompt using buildGeminiUserPrompt(context)
3. send both to Gemini in the request shape appropriate to @google/genai
4. keep the same validation and fallback behaviour from Prompt 6

Do not inline prompt text inside GeminiNarrator anymore.

Use promptBuilder.ts exclusively.

Do not change:

- model default
- fallback rules
- validation rules
- runtime selection logic

TESTS — tests/promptBuilder.test.ts

Import:

```
buildNarrationSceneContext from src/narration/narrationContext.ts
buildGeminiSystemPrompt from src/narration/promptBuilder.ts
buildGeminiUserPrompt from src/narration/promptBuilder.ts
initDefaultPlayerState from src/engine/state.ts
```

Also import any small types/helpers needed from existing files.

Tests:

buildNarrationSceneContext:

1. includes money_display using formatted currency
2. includes lute in inventory_summary.item_names by default
3. excludes blocked exits from location_summary.exits
4. builds University canon_era_context with 2–4 notes

buildGeminiSystemPrompt:

5. includes KKC canon instruction text
6. includes era_label and location_group
7. includes authoritative constraints text
8. does not include raw inventory item listing

buildGeminiUserPrompt:

9. includes location name and base description
10. includes time of day and day number

11. includes visible NPC names or "Visible people: none"
12. includes money display and inventory lines
13. includes engine_truth when present
14. ends with "Write the scene now."

Add 2 tests to existing tests/geminiNarrator.test.ts:

15. narrateScene() uses promptBuilder output rather than inline prompt text (mock promptBuilder functions if needed)
16. when promptBuilder returns valid strings and Gemini returns valid text, fallback is not used

ARCHITECTURE REMINDERS

These rules from Prompt 1 still apply:

- narration/* never reads from or writes to the DB
- narration/* never mutates PlayerState
- engine/* owns all state changes
- repl.ts saves state exactly once per turn
- No `any` types
- No external API calls outside the optional Gemini provider
- No unused imports or files

promptBuilder.ts is a narration-layer file.
It must not call the DB.
It must not decide gameplay outcomes.

SUCCESS CRITERIA

This prompt is complete when:

- NarrationSceneContext includes inventory and money summary
- NarrationSceneContext includes lightweight canon-era context
- Gemini prompt construction is moved out of GeminiNarrator into promptBuilder.ts
- the Gemini system prompt includes current era/location constraints
- the Gemini user prompt includes location, time, NPCs, inventory, money, and engine_truth
- blocked exits are not exposed in prompt context
- mystery bans remain intact
- local fallback still works exactly as before
- no new gameplay systems are introduced
- all new tests pass alongside earlier prompts' tests

- no refactoring of working Prompts 1–6 code beyond what is required

```
=====
END OF PROMPT 7
=====
```

The clean next move after this is **Prompt 8: Canon Registry module**, which was the original next step in your plan.

That feedback is mostly solid.

My read: **the direction is right**, but I would not accept every suggestion as-is.

The best changes to adopt are:

- make **engine_truth the primary event focus** when present
- add a stronger **style note** so the prose stays grounded
- avoid raw empty fields like **Injuries: None**
- keep Gemini away from **status / inventory** / other non-scene commands
- stop raw numerical hunger/fatigue from reaching the LLM

That fits the architecture you've already built, where the engine owns truth and narration only expresses it.

The one suggestion I would **not** adopt directly is adding **social_standing: string** to **PlayerState**. I would derive that inside **promptBuilder.ts** instead. Otherwise you start duplicating truth in state: the engine already has **university_social**, **eolian_standing**, trust values, and location context. A derived label is good; storing it in core state is not.

So my recommendation is:

Keep

- Prompt 7 as the right next move
- the system/user prompt split
- the compact scene context
- local fallback as a hard requirement

Change

- add **event-priority wording**
- add **style grounding**
- convert raw condition numbers to **descriptive labels** before prompt construction
- make prompt building return **nothing / local-only** for unsupported commands
- derive social flavour labels inside narration, not in core state

Here's the patch I'd apply to Prompt 7.

PATCHES FOR PROMPT 7

1. In `canonInstructions.ts`, add:

- If `engine_truth` contains an outcome, narrate that event as the primary focus of the response. Use the location and scene details as backdrop, not as the main event.
- Prose style: observational, restrained, slightly melancholy, materially grounded, attentive to physical sensation and social pressure. Prefer the feel of worn, lived reality over lyrical flourish.
- Do not mention raw numeric condition values. Use only the descriptive condition labels supplied in context.

2. In `src/types/index.ts`, update `NarrationSceneContext` so `player_summary` uses descriptive condition labels instead of raw numbers for LLM use:

```
export interface NarrationSceneContext {  
  command: string;  
  player_summary: {  
    era: string;  
    location_id: string;  
    time_of_day: TimeOfDay;  
    day_number: number;  
    money_drabs: number;  
    academic_rank: AcademicRank;  
    hunger_label: string;  
    fatigue_label: string;  
    warmth_label?: string;  
    injury_summary?: string;  
    social_standing_label?: string;  
  };  
  inventory_summary: {  
    item_names: string[];  
    money_display: string;  
  };  
  location_summary: {  
    id: string;  
    name: string;  
    description_base: string;  
    exits: string[];  
  };  
  npc_summary: {  
    names: string[];  
  };  
  canon_era_context: {  
    era_label: string;  
  };  
}
```

```

    location_group: string;
    notes: string[];
};
engine_truth: {
    movement_message?: string;
    sympathy_outcome?: string;
    social_outcome?: string;
    music_outcome?: string;
};
}

```

Notes:

- Do not remove the raw numbers from PlayerState.
- These labels are narration-context only.
- social_standing_label must be derived in narrationContext.ts from current reputation and location cluster. Do not store it in PlayerState.

3. In narrationContext.ts:

- Convert hunger/fatigue/warmth into labels before building context.
- If injuries are empty, omit injury_summary rather than sending "none".
- Derive social_standing_label modestly, e.g.:
 "an ordinary student at the University"
 "not yet known at the Eolian"
 "quietly better known in Imre"
 Keep this shallow and deterministic.
- If command is not one of the supported scene commands for Gemini, return null instead of a scene context.

Supported scene commands:

```

look / look around
go [direction] / bare direction
wait
use sympathy / bind ...
ask kilvin for work
work fishery
ignore ambrose
answer ambrose carefully
answer ambrose sharply
audition for pipes
play at eolian

```

4. Update promptBuilder.ts:

buildGeminiSystemPrompt(context):

- Include:
 "If engine_truth contains an outcome, narrate that event as the primary action of the scene. Treat the rest of the context as

supporting environment."

buildGeminiUserPrompt(context):

- Do not include raw numeric hunger/fatigue/warmth.
- Use descriptive labels only.
- If injury_summary is absent, omit the line entirely.
- If social_standing_label is present, include:
"Social standing: [label]"
- Add one compact note:
"Money has social weight relative to place. A few drabs mean different things at the University and in Imre."

5. Update actions.ts:

- Only call narrator.narrateScene(context) if buildNarrationSceneContext(...) returns a non-null context.
- If it returns null, use the existing local path.

6. Add tests to promptBuilder.test.ts:

15. buildNarrationSceneContext returns null for command 'inventory'
16. buildGeminiUserPrompt includes hunger/fatigue labels, not raw numbers
17. buildGeminiUserPrompt omits injury line when uninjured
18. buildGeminiSystemPrompt includes event-priority instruction

So my answer is: **yes, that feedback improves Prompt 7**, but I'd implement it in a way that preserves your architecture.

The most important design choice is this:

- **derive narration labels in the prompt builder**
- **do not push more presentation state into PlayerState**

That keeps the engine clean.

This refined version of Prompt 7 is architecturally superior to the initial suggestion. You have correctly identified a key "Vibe Coding" trap: ****state-bloat****.

By deriving labels in the narration layer rather than storing them in the PlayerState, you are keeping the engine's "brain" purely mathematical and the narrator's "voice" purely descriptive. This is the hallmark of a maintainable simulation.

Here are my thoughts on why this specific configuration is the "Gold Standard" for your project:

1. The "Command Gatekeeper" Logic

Implementing a null return for non-scene commands in `narrationContext.ts` is a brilliant move.

- * **Efficiency:** It saves API tokens and reduces latency on frequent actions like inventory or status.

- * **Reliability:** These commands are "Interface Actions," not "Story Actions." Local deterministic rendering is always more reliable for UI than an LLM trying to be clever with a list of items.

2. Event-Priority Narration

The "Action-Backdrop" instruction is the fix for the most common failure in AI-driven interactive fiction: the "Static Room" problem.

- * Without this, the LLM often sees the `location_summary` and spends 100 words on the architecture of the Fishery, while the `engine_truth.sympathy_outcome` (the actual explosion the player just caused) gets relegated to a single sentence at the end.

- * **Result:** The game will feel "reactive" rather than just "descriptive."

3. The "Label-Only" Constraint

Removing raw numbers (`hunger: 4`) and replacing them with `hunger_label: "your stomach is a hollow ache"` forces the LLM to remain in-character.

- * Sticking to engine-supplied labels prevents the LLM from inventing its own severity levels (e.g., narrating that you are "dying of starvation" when you are only moderately hungry).

4. Derived Social Flavour

Deriving `social_standing_label` inside the narration layer based on `location_id` and `reputation` is exactly how a human DM works. They look at the stats and then decide the "vibe" of the room.

- * **Architecture Win:** If you ever change how Reputation is calculated in Prompt 45, you only have to update the mapping in one place (`narrationContext.ts`) and the "voice" of the game updates automatically.

5. Deterministic Mapping

One small implementation detail to watch for in `narrationContext.ts`: ensure the mapping from numbers to labels is **deterministic and tiered**.

For example:

- * **Fatigue 0-2:** "You feel sharp and alert."

- * **Fatigue 3-5:** "A weight sits behind your eyes."

- * **Fatigue 6+:** "Your thoughts move like thick honey."

Final Verdict

This is the most robust version of the plan yet. It preserves the **Engine as Source of Truth** principle while giving the **Narrator** the high-quality, filtered data it needs to produce Rothfuss-tier prose.

With these patches, Prompt 7 is ready. It sets a perfect stage for **Prompt 8: The Lore Registry**, as the Lore Registry will essentially just be another "label and note injector" that plugs into this same `promptBuilder.ts` architecture.

You are ready to proceed with Prompt 7.